

halfedge：测地线距离模块设计文档

src/geodesics.rs

halfedge 库

2026-07-05

目录

1 模块定位	2
2 Heat Method 算法	2
2.1 Step 1: 时间步 $\tau = h^2$	2
2.2 Step 2: 热流扩散	3
2.3 Step 3: 单位负梯度	3
2.4 Step 4: 散度与 Poisson 求解	3
2.5 Step 5: 距离场偏移校正	3
3 算法流程图	4
4 离散算子	4
4.1 离散拉普拉斯（余切权重）	4
4.2 面梯度（FEM 形函数）	4
4.3 面散度（弱形式积分）	5
5 最短路径回溯	5
6 Dijkstra 最短图距离	5
6.1 最小堆实现	5
6.2 算法流程	6
6.3 多源 Dijkstra	6
6.4 路径回溯: dijkstra_with_parent	6
7 多源 Heat Method	7
8 Heat Method 与 Dijkstra 对比	7
9 退化守卫	7
10 使用示例	8
11 测试覆盖	8

1 模块定位

`geodesics.rs` 在 `geometry.rs` / `linalg.rs` 之上实现 **Heat Method** (Crane, Weischedel, Wardetzky 2013), 通过求解两个稀疏线性系统高效计算三角网格上从源顶点出发的测地线距离, 并提供沿距离场梯度回溯最短路径的能力。

类别	API	语义
距离场 (Heat Method)	<code>geodesic_distance_from_vertex</code>	Heat Method: 从 <code>source</code> 出发计算所有顶点的测地线距离; 返回顺序对应 <code>mesh.vertex_ids()</code> ; 求解失败返回 <code>None</code> 。
路径回溯	<code>shortest_path</code>	沿距离场梯度回溯最短路径: 返回从 <code>target</code> 到源顶点的顶点序列 (含两端)。
距离场 (Dijkstra)	<code>dijkstra_geodesic</code>	单源 Dijkstra 最短图距离 (边长权值), 不依赖线性求解器
距离场 (Dijkstra 多源)	<code>dijkstra_multi_source_geodesic</code>	多源 Dijkstra: 每个顶点到最近源点的图距离
路径回溯 (Dijkstra)	<code>dijkstra_with_parent</code>	Dijkstra + 父节点回溯数组, 用于精确路径重建
路径 (Dijkstra 精确)	<code>dijkstra_shortest_path</code>	返回 <code>source</code> 到 <code>target</code> 的最短顶点序列 (含两端)
距离场 (多源 Heat Method)	<code>multi_source_geodesic</code>	多源 Heat Method: 每个顶点到最近源点的测地线距离; 源点强制为 0 消除数值误差

2 Heat Method 算法

Heat Method 把**非线性**的测地线距离计算转化为**两个线性系统**: 先用热流方程扩散初始脉冲得到光滑梯度方向, 再用 Poisson 方程从散度场恢复距离。其核心洞察是: 热流梯度的方向与测地线梯度方向几乎一致, 且方向对参数不敏感。

2.1 Step 1: 时间步 $\tau = h^2$

设网格平均边长为 h , 时间步取

$$\tau = h^2, \quad h = \frac{1}{|E|} \sum_{e \in E} L(e).$$

取 h^2 的理由: 使热流在每条边上的扩散量 $\tau \cdot \frac{1}{h^2} \sim O(1)$, 既不会过快抹平局部信息, 也不会过慢导致数值噪声。代码中若 $|E| = 0$, 回退 $\tau = 1$ 。

2.2 Step 2: 热流扩散

求解线性系统

$$(M + \tau L) u_\tau = M u_0,$$

其中 M 为 lumped mass (顶点 Voronoi 面积对角矩阵), L 为余切拉普拉斯, u_0 为初始热源。初始热源取源顶点 s 处的单位积分脉冲:

$$u_0[i] = \begin{cases} 1/M_{ss}, & i = s, \\ 0, & \text{otherwise.} \end{cases}$$

等价于热传导方程 $\frac{\partial u}{\partial t} = \Delta u$ 在时间 τ 后的解。代码使用共轭梯度法 (conjugate_gradient) 求解, 最大迭代数 100 n , 相对残差 10^{-6} , 并预先调用 regularize_diagonal 加入 10^{-10} 对角正则化以保证 SPD 性。

2.3 Step 3: 单位负梯度

对每个三角面 f , 由分段线性基函数求 u_τ 的常梯度 $\nabla u_\tau|_f$, 再归一化并取负:

$$X_f = -\frac{\nabla u_\tau|_f}{\|\nabla u_\tau|_f\|}.$$

取负原因: 热流从高温 (源) 流向低温 (远端), ∇u_τ 指向远离源, 而测地线最短路径应朝向源前进, 故取负。退化面 ($\|\nabla u_\tau\| < 10^{-10}$) 置 $X_f = \vec{0}$, 避免除零。

2.4 Step 4: 散度与 Poisson 求解

将单位负梯度场 X 投影回顶点散度 (弱形式):

$$(\nabla \cdot X)_i = \sum_{f \ni i} A_f \cdot (X_f \cdot \nabla \varphi_i|_f),$$

其中 φ_i 是顶点 i 的分片线性基函数。再求解 Poisson 方程

$$L \phi = \nabla \cdot X,$$

L 与 Step 2 共用余切拉普拉斯。同样使用共轭梯度法求解。

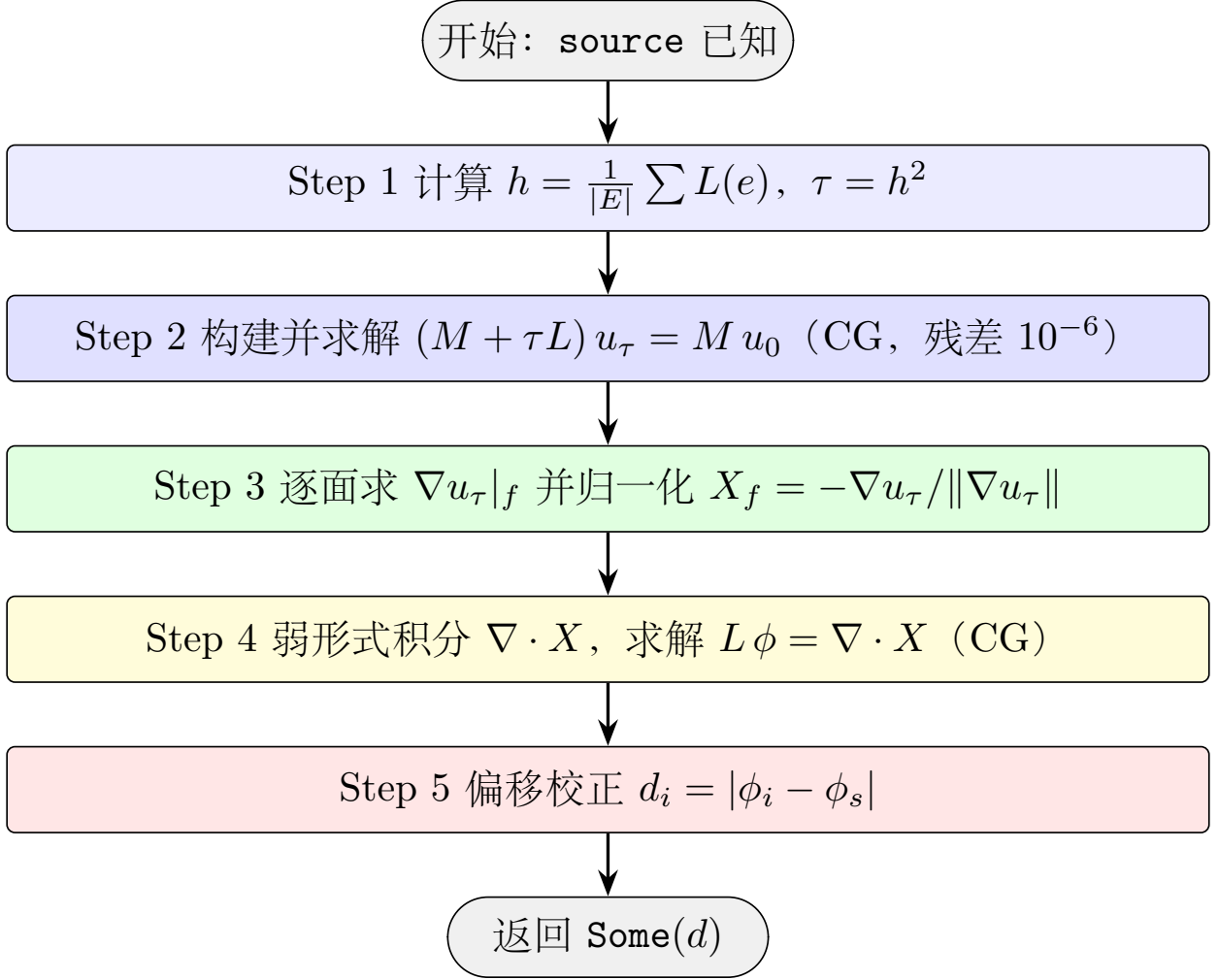
2.5 Step 5: 距离场偏移校正

Poisson 解 ϕ 与真实测地线距离 d 仅差一个常数 (Δ 的核空间)。取源顶点 s 处为零参考点:

$$d_i = |\phi_i - \phi_s|.$$

取绝对值是防止数值误差导致个别顶点 ϕ_i 略小于 ϕ_s 。校正后 $d_s = 0$, 其余 $d_i \geq 0$ 。

3 算法流程图



4 离散算子

4.1 离散拉普拉斯（余切权重）

对相邻顶点 i, j , 设边 ij 的两个对角为 α_{ij}, β_{ij} , 则

$$L_{ij} = -\frac{1}{2}(\cot \alpha_{ij} + \cot \beta_{ij}), \quad L_{ii} = -\sum_{j \neq i} L_{ij}.$$

边界边只有一个对角, 使用单边余切。实现中仅累加 $w > 0$ 的项（非钝角三角形贡献）以避免负权重导致矩阵非 SPD。Lumped mass M_{ii} 取顶点周围面面积的 $1/3$ 之和, 退化时钳为 10^{-14} 。

4.2 面梯度（FEM 形函数）

对三角面 f 的三顶点 i, j, k , 分片线性函数 $u|_f = u_i \varphi_i + u_j \varphi_j + u_k \varphi_k$ 的常梯度为

$$\nabla u|_f = \frac{1}{2A_f} \hat{n}_f \times (u_i \vec{e}_i + u_j \vec{e}_j + u_k \vec{e}_k),$$

其中 $\vec{e}_i = \vec{p}_k - \vec{p}_j$ 是顶点 i 所对的边向量, A_f 是面面积, $\hat{n}_f$ 是面法向。

4.3 面散度（弱形式积分）

向量场 X 在顶点 i 处的散度（弱形式）为

$$(\nabla \cdot X)_i = \sum_{f \ni i} A_f \cdot (X_f \cdot \nabla \varphi_i|_f), \quad \nabla \varphi_i|_f = \frac{1}{2A_f} \hat{n}_f \times \vec{e}_i,$$

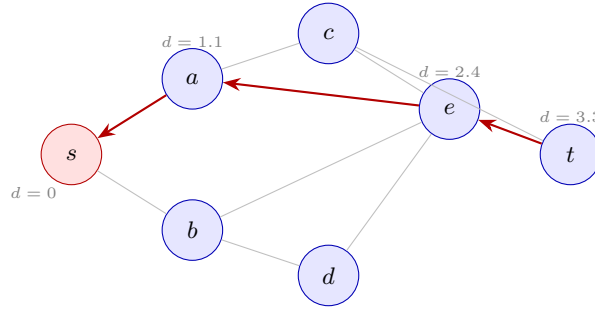
等价地写作

$$(\nabla \cdot X)_i = \frac{1}{2} \sum_{f \ni i} X_f \cdot (\hat{n}_f \times \vec{e}_i).$$

代码中 $\nabla \varphi_i|_f$ 直接用 $\hat{n}_f \times \vec{e}_i / (2A_f)$ 计算，再乘以 A_f 与 X_f 的点积，与上式一致。

5 最短路径回溯

`shortest_path` 在已计算好的距离场 d 上，沿**贪心梯度下降**方向回溯：当前顶点 v 的邻域中，选择 d 最小且严格小于当前距离的顶点 v' ，跳转至 v' ，重复直至到达源点（ $d < 10^{-10}$ ）或陷入局部极小（无更小邻点）。步数上界 $2|V|$ 防止死循环。



回溯方向： $t \rightarrow e \rightarrow a \rightarrow s$ （沿 d 严格递减）

贪心策略的局限：若距离场存在局部极小（数值误差或网格各向异性），可能在到达源点前提前终止；上界 $2|V|$ 保证不进入死循环。返回序列包含两端顶点，从 `target` 起到源点止。

6 Dijkstra 最短图距离

`dijkstra_geodesic` 使用经典 Dijkstra 算法计算**图最短距离**：以边长（欧氏长度）为权值，求源点到所有顶点的最短路径长度。相比 Heat Method，Dijkstra 路径只能沿网格边行进，结果为**图距离上界**，但实现简单、不依赖线性求解器，且结果**精确**（无数值误差）。

6.1 最小堆实现

Rust 标准库 `BinaryHeap` 是最大堆，通过反转 `Ord` 比较方向实现最小堆：

```
1 #[derive(Clone, Copy, Debug, PartialEq)]
2 struct QueueEntry { dist: f64, vertex: usize }
3
4 impl Ord for QueueEntry {
5     fn cmp(&self, other: &Self) -> Ordering {
6         // BinaryHeap
7         other.dist.partial_cmp(&self.dist).unwrap_or(Ordering::Equal)
```

```

8         .then_with(|| other.vertex.cmp(&self.vertex))
9     }
10 }

```

6.2 算法流程

1. 初始化 $\text{dist}[\text{source}] = 0$ ，其余为 $+\infty$ ；
2. 将 $(0, \text{source})$ 入堆；
3. 循环弹出堆顶 (d, v) ：
 - 若 $d > \text{dist}[v]$ ，跳过（已通过更短路径访问过）；
 - 遍历 v 的邻居 u ，若 $d + \|p_u - p_v\| < \text{dist}[u]$ ，更新 $\text{dist}[u]$ 并入堆 $(d + \|p_u - p_v\|, u)$ ；
4. 堆空时终止。

复杂度 $O(|E| \log |V|)$ 。

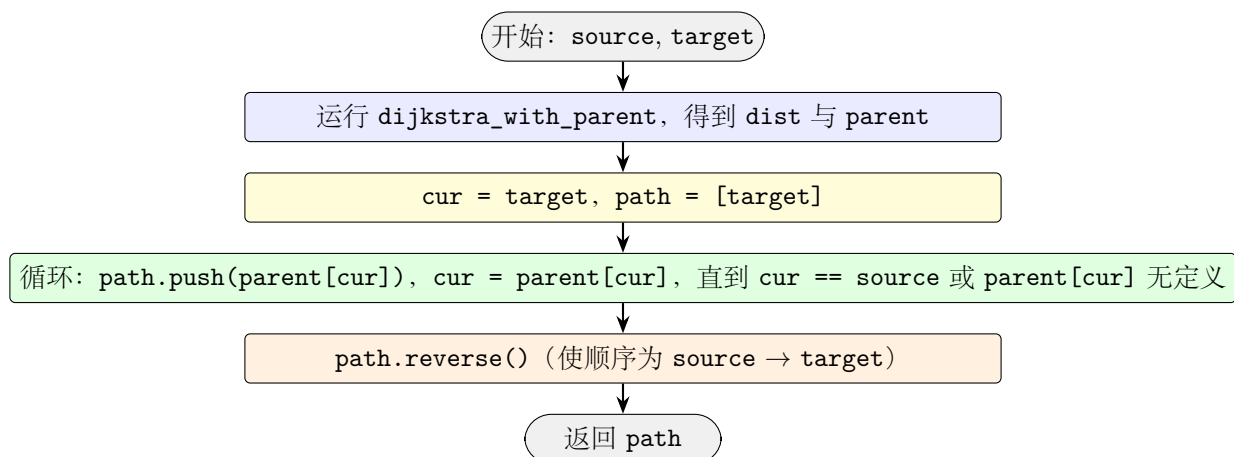
6.3 多源 Dijkstra

`dijkstra_multi_source_geodesic` 将所有源点以距离 0 同时入堆，自然得到每个顶点到最近源点的距离。

6.4 路径回溯：dijkstra_with_parent

`dijkstra_with_parent` 在 Dijkstra 同时维护 `parent` 数组：每次松弛 $\text{dist}[u]$ 时记录 $\text{parent}[u] = v$ 。返回 $(\text{dist}, \text{parent})$ 二元组，便于精确路径重建。

`dijkstra_shortest_path` 在此基础上从 `target` 沿 `parent` 链回溯到 `source`，得到精确最短顶点序列。



7 多源 Heat Method

`multi_source_geodesic` 将 Heat Method 扩展到**多源**场景：计算每个顶点到最近源点的测地线距离。实现上仅修改初始热源 u_0 ：

$$u_0[i] = \begin{cases} 1/M_{ii}, & i \in S(\text{任一源点}), \\ 0, & \text{otherwise.} \end{cases}$$

其余步骤（热流扩散、梯度归一化、Poisson 求解、偏移校正）与单源完全相同。

源点数值误差处理：Heat Method 在源点处存在数值误差，单源时通过 ϕ_s 偏移校正消除；多源时各源点误差不一致，故在偏移校正后**显式将每个源点距离置零**：

$$\forall s \in S: d_s \leftarrow 0.$$

8 Heat Method 与 Dijkstra 对比

属性	Heat Method	Dijkstra
路径类型	测地线（可跨面）	图路径（仅沿边）
精度	近似（数值求解）	精确（无误差）
依赖	稀疏线性求解器（CG）	仅需最小堆
复杂度	$O(k \cdot n)$ （CG 迭代）	$O(E \log V)$
源点误差	有（偏移校正后 ≈ 0 ）	无
多源扩展	修改 u_0 + 显式置零	多源同时入堆

9 退化守卫

函数	退化场景	处理
<code>geodesic_distance_from_vertex</code>	空网格 ($ V = 0$)	返回 <code>Some(Vec::new())</code>
<code>geodesic_distance_from_vertex</code>	<code>source</code> 不在 <code>v_idx</code> 中	返回 <code>None</code> （? 传播）
<code>geodesic_distance_from_vertex</code>	$ E = 0$	时间步回退 $\tau = 1$
<code>geodesic_distance_from_vertex</code>	共轭梯度未收敛	CG 返回 <code>None</code> ，函数返回 <code>None</code>
<code>build_laplacian_and_mass</code>	顶点面积 $< 10^{-14}$	钳为 10^{-14} ，防除零
<code>build_laplacian_and_mass</code>	余切权重 $w \leq 0$	跳过该边，避免破坏 SPD
<code>compute_face_gradients</code>	面积 $< 10^{-14}$ （退化面）	跳过该面
Step 3 归一化	$\ \nabla u_\tau\ < 10^{-10}$	$X_f = \vec{0}$
<code>shortest_path</code>	<code>target</code> 无效或越界	返回 <code>Vec::new()</code>
<code>shortest_path</code>	陷入局部极小	提前终止，返回已收集路径
<code>shortest_path</code>	步数达上界 $2 V $	终止循环，防止死循环

10 使用示例

```
1 use halfedge::geodesics::{geodesic_distance_from_vertex, shortest_path};
2 use halfedge::test_util::build_icosphere;
3
4 let mesh = build_icosphere(2);
5 let vertices: Vec<_> = mesh.vertex_ids().collect();
6 let source = vertices[0];
7 let target = vertices[vertices.len() / 2];
8
9 // 1.      source
10 let dist = geodesic_distance_from_vertex(&mesh, source)
11     .expect("heat method ok");
12
13 //      0
14 assert!(dist[0] < 1e-6);
15
16 // 2.      target
17 let path = shortest_path(&mesh, &dist, target);
18 assert!(path.len() >= 2);
19 assert_eq!(*path.first().unwrap(), target);
20 assert_eq!(*path.last().unwrap(), source);
```

11 测试覆盖

测试名	验证点
<code>test_geodesic_self_distance</code>	在 icosphere 上调用 <code>geodesic_distance_from</code> 断言返回 <code>Some</code> ；源顶点距离 $< 10^{-6}$ ；至少一个非源顶点距离 > 0 。
<code>test_geodesic_monotonicity</code>	在 icosphere 上对前若干非源顶点做 sanity check，验证距离场单调性。至少存在一个顶点距离更小，验证距离场单调性。
<code>test_dijkstra_self_distance_zero</code>	源点到自身 Dijkstra 距离 = 0
<code>test_dijkstra_symmetric_on_icosphere</code>	icosphere 上 $d(s \rightarrow t) \approx d(t \rightarrow s)$ (容差 10^{-6})
<code>test_dijkstra_distance_upper_bounds_heat_method</code>	Dijkstra 与 Heat Method 的最大距离在 3 倍以内 (距离 vs 测地线)
<code>test_dijkstra_multi_source</code>	多源 Dijkstra: 每个顶点距离 $\leq$ 单源距离；源点距离 = 0
<code>test_dijkstra_shortest_path_self</code>	<code>source == target</code> 时返回 <code>[source]</code>
<code>test_dijkstra_shortest_path_to_neighbor</code>	邻居路径长度为 2 (<code>[source, target]</code>)
<code>test_dijkstra_shortest_path_consistency_with_distance</code>	路径长度与距离一致: $\text{dist}[\text{target}] \approx \sum \ p_{i+1} - p_i\ $
<code>test_multi_source_geodesic_sources_zero</code>	多源 Heat Method 所有源点距离 $< 10^{-6}$ (强制)
<code>test_multi_source_geodesic_le_single_source</code>	多源最大距离 $\leq$ 单源最大距离 (容差 10^{-6})

`src/geodesics.rs` 共 11 个单元测试，全库共 394 个。